IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 21 July 2025, accepted 15 September 2025, date of publication 19 September 2025, date of current version 2 October 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3612038

RESEARCH ARTICLE

LiquidGAN: Integrating Liquid Neural Networks and Neural ODEs for High-Fidelity Thermal Image Synthesis Under Data Scarcity

S. SARATH AND JYOTHISHA J. NAIR, (Senior Member, IEEE)
Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri, India
Corresponding author: S. Sarath (saraths@am.amrita.edu)

ABSTRACT The growing importance of thermal imaging in a variety of fields, including medical diagnostics, security, and environmental monitoring, emphasizes the critical need for efficient deep learning models capable of analysing thermal data. The training of these algorithms is much limited by the absence of suitably annotated thermal imaging datasets. We propose LiquidGAN, a novel Generative Adversarial Network (GAN) architecture which integrates the Neural Ordinary Differential Equations (ODEs) and Liquid Neural Networks (LNNs) to address the scarcity of annotated thermal imaging datasets. LiquidGAN employs a fixed-step Runge-Kutta (RK4) solver to refine the latent representations, enabling continuous image transformations through an encoder-ODE-decoder generator. The LNN-based discriminator adaptively models the subtle, and the dynamic temperature features, hence improving the ability to distinguish the real images from synthetic images. By integrating ODE-driven liquid dynamics across both the generator and discriminator, LiquidGAN effectively captures the dynamic temperature variations while stabilizing adversarial training under the limited data conditions. Moreover, Automatic Mixed Precision (AMP) and gradient scaling are used in our work to further improve the computational efficiency. Experimental results on multiple thermal imaging datasets, including those related to surveillance, healthcare, and precision agriculture, suggest that LiquidGAN offers a promising direction for generating high-fidelity synthetic thermal images under data-constrained conditions.

INDEX TERMS Generative adversarial networks, liquid neural networks, neural ordinary differential equations, thermal imaging, data scarcity, data augmentation, continuous dynamics, mixed precision training.

I. INTRODUCTION

Thermal imaging is widely used in medical diagnostics, autonomous navigation, surveillance, precision agriculture, and industrial monitoring. However, the lack of high-quality annotated datasets effectively reduces the efficiency of the deep learning models in thermal image analysis. Unlike natural image datasets like ImageNet, thermal datasets are often smaller in size and less diverse due to the specialized sensors needed for data collection and expertise required for accurate labeling. This scarcity leads to poor generalization, increasing models' susceptibility to overfitting and performance degradation in real-world scenarios.

People extensively use data augmentation to artificially expand the training dataset and alleviate data shortage problems. Data augmentation involves applying transformations to existing images to increase their variability without altering their underlying semantics. Visible spectrum images have shown success with conventional augmentation methods including flipping, rotation, cropping, and color jittering. But thermal images have special properties such as temperature dependent pixel intensities, sensor noise, and emissivity variations that make traditional augmentation techniques less useful. Unlike RGB images, where augmentations such as hue shifts or saturation adjustments preserve important features, comparable alterations in thermal images can distort temperature distributions, thereby producing unrealistic and misleading representations. For thermal image augmentation to be effective, it must preserve the fundamental thermal properties

The associate editor coordinating the review of this manuscript and approving it for publication was Rajeswari Sundararajan.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

while introducing diversity in the dataset. Some possible augmentation techniques for thermal images include:

- **Geometric transformations**: Geometric transformations like flipping, rotation, and scaling are very useful but must be carefully applied, as excessive transformations may create artifacts in spatial temperature distributions.
- **Histogram equalization and contrast enhancement**: Since thermal images often have low contrast, adaptive histogram equalization can improve feature visibility without distorting thermal gradients.
- **Noise addition and blur simulations**: Simulating real-world sensor noise and blur effects can help models become more robust to imperfect sensor conditions.
- **Generative augmentation with GANs**: GANs can be used to synthesize new thermal images that maintain realistic temperature distributions while increasing dataset diversity.

Although GAN-based augmentation techniques has considerable capability, conventional GAN architectures face specific challenges in the context of thermal imaging. Common issues such as training instability, mode collapse, and limited control over the fine-grained feature transformations restrict their performance. Standard convolutional generators that rely on fixed-weight transformations, often fails to capture the continuous and gradual variations of real-world temperature distributions. This leads to the generation of unrealistic synthetic images. Additionally, CNN-based discriminators are rigid and struggle to adapt to the nuances of thermal textures, limiting their ability to guide the generator effectively during adversarial training.

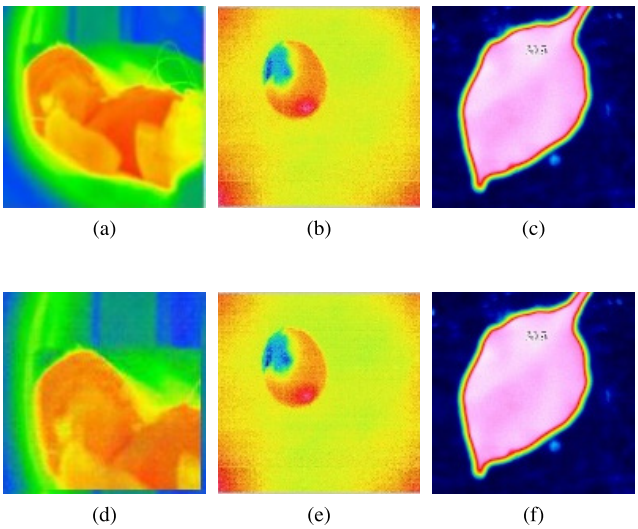

FIGURE 1. Real thermal images are shown in (a), (b), and (c), while (d), (e), and (f) shows the corresponding synthetic images generated by the proposed LiquidGAN model.

To address these limitations, we propose a novel GAN architecture - LiquidGAN that integrates Neural ODEs into the generator and LNNs into the discriminator. LiquidGAN's generator is capable to achieve smoother and realistic variations of synthesized images, since it models latent space transformations as continuous-time dynamical systems using ODEs.The LNN-based discriminator on the other hand enhances its ability to capture fine-grained thermal details by making use of adaptive weight dynamics.

Our approach is suitable for real-world thermal imaging applications including neonatal healthcare monitoring, agricultural inspection, and security surveillance. Extensive experimentation across multiple datasets, shows that our approach consistently outperforms conventional GAN architectures in terms of image quality, diversity, and training stability—especially under conditions of data scarcity.

LiquidGAN applied to different thermal imaging datasets shows that it outperforms the traditional GAN architectures like Deep Convolutional Generative Adversarial Network (DCGAN) [14] and Wasserstein GAN (WGAN-GP) [15] in terms of image quality, variety, and training stability. Our method guarantees that augmented thermal images keep their basic characteristics; hence, they are appropriate for enhancing deep learning models in several thermal imaging applications. The LNN discriminator changes its internal state over time, unlike conventional CNN-based discriminators with stationary weight updates. Hence, it enhances adversarial training stability and resilience to the distribution changes. This dynamic property helps prevent mode collapse and lets the discriminator learn more meaningful representations of generated thermal images. LiquidGAN addresses the limitations of existing augmentation approaches by creating a more robust and efficient framework for synthetic thermal data production.

II. PROBLEM ADDRESSED

In spite of the recent advancements in GANs, their direct application to thermal image synthesis has several constraints. The GAN architectures are not suitable for the thermal images for the following three reasons:

1) **Training Instability:** GAN training is unstable, often leading to mode collapse where the generator produces a limited variety of outputs.
2) **Lack of Fine-Grained Control:** For thermal images which are represented as continuous and gradual temperature variations, conventional GAN generators which rely on fixed-weight convolutional layers is insufficient.
3) **Rigid Discriminators:** Thermal images contains subtle and temperature dependent features. Traditional CNN-based discriminators use stationary weight updates and static feature extraction techniques which are unsuitable for thermal images and hence reduces their effectiveness in guiding the generator during adversarial training.

Hence, we propose LiquidGAN, a novel architecture designed by integrating Neural ODEs and LNNs into the GAN framework.

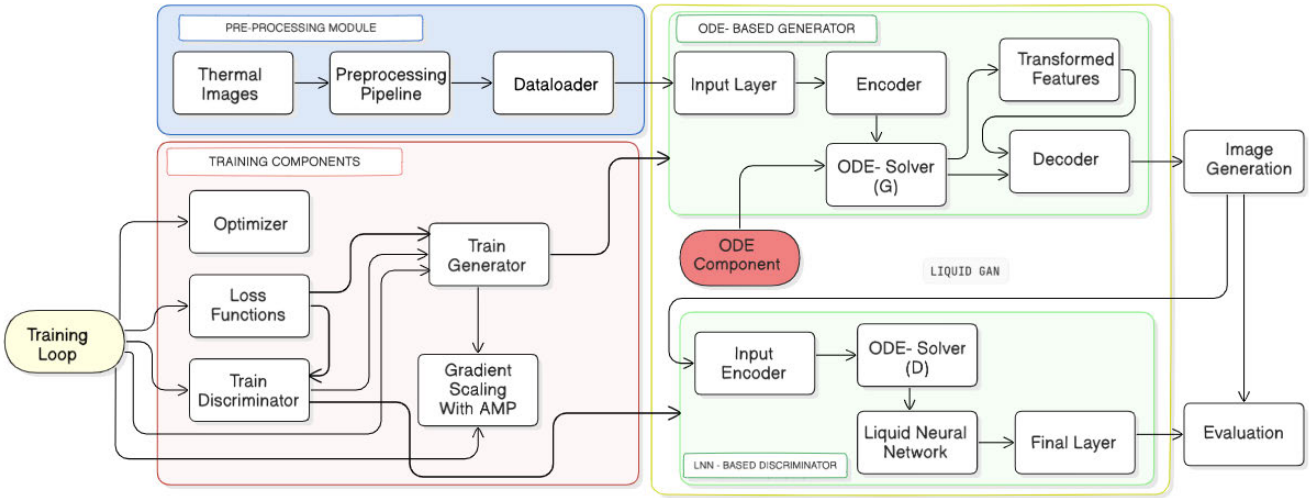

FIGURE 2. System architecture.

III. RELATED WORK

Generative Adversarial Networks, introduced by Goodfellow et al. [1], have revolutionised data generation through their adversarial framework, which consists of both a generator and discriminator. The generator synthesises the data while the discriminator distinguishes between the real and synthetic samples, driving the generator to create increasingly realistic outputs. GANs have demonstrated remarkable success across a variety of domains, including image synthesis, domain adaptation, and data augmentation.

Despite their success, most of the traditional GANs [11] face inherent limitations such as mode collapse, where the generator produces limited variations, and training instability due to the adversarial nature of the learning process. Advances like Wasserstein GANs (WGANs) [2] and StyleGANs [3] have addressed these issues by refining loss functions and improving generator architecture, enabling higher-quality and more diverse outputs. However, when applied to complex domains such as thermal imaging, conventional GANs struggle to capture subtle data nuances, highlighting the need for more sophisticated discriminator mechanisms to evaluate fine-grained variations effectively.

Thermal imaging, with applications spanning medical diagnostics [10], security, and industrial monitoring, has seen growing interest. However, the limited availability of annotated thermal datasets remains a significant bottleneck in advancing machine learning models for this domain. GANs have been extensively used for data augmentation, generating synthetic thermal images to address dataset scarcity. Mizginov et al. [4] focused on the development of a method for synthesising thermal imaging images based on generative adversarial neural networks. Sarath and Nair [5] used GANs to improve infrared thermal images to find respiratory syndromes in newborns. The results showed that synthetic images could make detection models more accurate. Aloysius and Geetha [12] had done a detailed review of all the CNN networks helping to decide which CNN suits best in a scenario. Mohan et al. [13] had discussed the use of a cyclic model that could be used with isochromatic images. Aloysius and Geetha [16] had proposed a weighted average ensemble of low, intermediate and high resolution networks which improved the accuracy of the model. Remya et al. [17] had used data augmentation to improve the results of vision transformers for cotton pest detection.

While these efforts highlight the potential of GANs in thermal data augmentation, conventional GAN architectures often fail to capture the unique characteristics of thermal data, such as subtle temperature gradients and spatial object contours. These shortcomings result in synthetic images that lack the desired realism or diversity. Incorporating advanced components, such as Neural ODEs [6], into the discriminator architecture offers a promising avenue for overcoming these challenges by enabling nuanced evaluations of synthetic thermal data.

Chen et al. [6] represent a paradigm shift in deep learning by treating transformations as solutions to differential equations over continuous time. This approach eliminates the need for discrete layers, allowing for smoother and more flexible learning trajectories while reducing memory overhead. Neural ODEs have been particularly impactful in generative models, where continuous latent space transformations improve the quality and diversity of generated samples. Grathwohl et al. [7] extended neural ODEs to scalable generative flows, demonstrating their ability to create complex data representations with reversible dynamics. In the context of GANs, integrating neural ODEs into the discriminator provides a significant advantage. By enabling continuous feature space transformations, neural ODE-based discriminators dynamically adapt to subtle data variations, such as those present in thermal images. This flexibility

not only makes the discriminator more sensitive to subtle patterns, but it also makes GAN training more stable by dealing with problems like mode collapse and oscillations in adversarial loss dynamics.

LNNs are a biologically inspired class of architectures that offer dynamic adaptability, making them particularly suited for complex and temporal data. Unlike conventional deep networks with fixed-layer configurations, LNNs rely on continuously evolving internal states influenced by input stimuli. Hasani et al. [8] introduced liquid time-constant networks, which use differential equations to model neuron behaviours, enabling improved robustness in dynamic environments such as robotics and time-series analysis. The adaptive nature of LNNs stems from their reliance on neural ODEs, which allow for time-continuous and fluid transformations. This capability makes LNNs a promising candidate for tasks requiring high flexibility, such as thermal data generation, where complex variations in temperature and spatial features must be accurately represented. Although relatively underexplored in generative modelling, the integration of LNN principles into GAN discriminators can significantly enhance the evaluation of generated data, enabling better alignment with real-world distributions and ensuring the generation of high-quality synthetic thermal images. Recent advancements in dynamic neural architectures have introduced mechanisms that adapt model parameters based on input-specific characteristics. Sun et al. [18] introduced Gaussian Dynamic Convolution (GDC), which achieves efficient single-image segmentation by allowing convolutional filters to vary dynamically with spatial context.

IV. METHODOLOGY

Our paper proposes a robust framework with five important modules to address the problems of thermal data augmentation. The first module is the data preprocessing pipeline. It normalises the temperature gradients in the input thermal images and gets them ready for model training by making the features more consistent. The second module is the neural ODE-based generator. It uses an encoder-decoder architecture with an ODE-driven latent transformation for enhanced feature dynamics. The third module is the liquid neural network-based discriminator. It is a convolutional discriminator that utilises neural dynamics via ODEs for better robustness. The fourth module is the training optimization module, which utilizes mixed precision training with Automatic Mixed Precision (AMP) and Runge-Kutta ODE solvers to maintain stability and efficiency in the model. The fifth module has periodic storage of generated images and makes a qualitative analysis of augmentation performance.

A. DATA PREPROCESSING

The data preprocessing module prepares the input thermal data for effective utilization within the GAN framework. This ensures that the data is normalised, resized, and augmented, which helps establish consistency and improves the comparison of real and synthetic images by the model. Temperature values are normalized to a standard range $[-1, 1]$, to ensure uniform presentation and prevent numerical instability during the training time. All the thermal images are resized to the given standard dimensions to ensure compatibility with the model architecture and preserve the spatial consistency throughout the dataset. The detailed step-by-step process is shown in Algorithm 1. A parallelized PyTorch dataloader is also implemented with the setting *pin_memory* = *True* for getting faster GPU memory transfers.

Given below is a detailed breakdown of its mathematical formulation:

Algorithm 1 Data Preprocessing Module

Input: Raw thermal images, Target dimensions (H, W)
Output: Preprocessed thermal images $I_{\text{Normalised}}$

1: **Begin**
2: **for** each raw thermal image I in dataset **do**
3: Convert I to RGB format
4: Resize I to target dimensions (H, W)

$$I_{\text{resized}} = \text{Resize}(I, H, W)$$

5: Apply center cropping:

$$I_{\text{cropped}} = \text{CenterCrop}(I_{\text{resized}})$$

6: Convert I_{cropped} to tensor representation
7: Normalize pixel values to range $[-1, 1]$

$$I_{\text{Normalised}} = \frac{I_{\text{cropped}} - 0.5}{0.5}$$

8: **end for**
9: **End**

B. NEURAL ODE-BASED GENERATOR

A neural ODE-based generator has an encoder-ODE-decoder architecture that forms the core of this proposed methodology. It helps in creating realistic thermal images while retaining the spatial and structural consistency. Compared to the traditional generators, the encoders extract spatial features using convolutional layers with the ReLU activation function and are then transformed over a period using an ODE solver. For stable continuous-time latent feature transformation, we utilized a fixed-step fourth-order Runge-Kutta (RK4) solver. RK4 was selected due to its balance between numerical accuracy and computational efficiency, as observed in prior ODE-based generative models. With the help of transposed convolutional layers and Tanh activation functions, the decoder reconstructs the thermal images, enabling the controlled pixel intensity distribution. The generator gets dynamic and structured variations for producing more realistic augmented images by incorporating the ODE-based feature transformation.

1) THE ENCODER

The encoder module takes a preprocessed thermal image $I \in \mathbb{R}^{H \times W \times C}$ and extracts the spatial and contextual features as given in equation 1.

$$h_0 = E(I, \theta_e) \tag{1}$$

where E is the encoding function, θ_e are the learnable parameters of the encoder, and h_0 is the latent feature representation. The transformation basically involves a sequence of convolutional operations, each followed by a non-linear activation function, as shown in equation 2.

$$h_i = \text{ReLU}(W_i * h_{i-1} + b_i), \quad i = 1, \ldots, N \tag{2}$$

where h_i is the output (feature map) of the activation function at layer i, h_{i-1} is the output of the previous layer. W_i, b_i represents the learnable weights and biases, and ReLU is the activation function used.

2) ODE-BASED LATENT FEATURE PROGRESSION

The encoded feature representation h_0 takes a temporal evolution using an ODE, as shown in the given equation 3.

$$\frac{dh}{dt} = f(h, t, \theta_{\text{ODE}}) \tag{3}$$

where $h(t)$ represents the latent feature at time t., θ_{ODE} are the parameters controlling the latent progression, and $f(h, t, \theta_{\text{ODE}})$ are the learnable neural progression. An ODEINT function calculates the progression, as shown in equation 4.

$$h_T = \text{ODEINT}(f, h_0, t, \text{method} ='' rk4'') \tag{4}$$

where h_T is the transformed latent state at time T, $t = [0, T]$ shows a distinct time sequence, and the Runge-Kutta (RK4) method is used for stable numerical integration.

The Runge-Kutta solver approximates h_T for any given ODE function f using four intermediate steps as shown in the equations 5(a) to 5(e).

$$k_1 = f(h, t) \tag{5a}$$

$$k_2 = f\left(h + \frac{\Delta t}{2} k_1, t + \frac{\Delta t}{2}\right) \tag{5b}$$

$$k_3 = f\left(h + \frac{\Delta t}{2} k_2, t + \frac{\Delta t}{2}\right) \tag{5c}$$

$$k_4 = f\left(h + \Delta t k_3, t + \Delta t\right) \tag{5d}$$

$$h_{t+\Delta t} = h_t + \frac{\Delta t}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right) \tag{5e}$$

where the values k_1, k_2, k_3, k_4 are the intermediate slopes used to compute the next value of h. They compute the function's behaviour at multiple points within the small time step Δt and finally the weighted sum.

3) THE DECODER

The decoder module reconstructs the image from the transformed latent representation as shown in equation 6.

$$\hat{I} = f_{\text{dec}}(h_T, \theta_{\text{dec}}) \tag{6}$$

where f_{dec} is the decoder function, θ_{dec} are the decoder parameters, and $\hat{I}$ is the reconstructed image. The decoder performs a sequence of inverse transformations, which includes transposed convolutions, upsampling, and activation functions. The Tanh activation function is applied in the final layer, which produces the output values in the range $[-1, 1]$.

4) LOSS FUNCTION AND OPTIMISATION OF MODEL

The generator is trained by combining adversarial and reconstruction losses of the model. The equation 7 shows the calculation of the loss.

$$L_G = \mathbb{E}_{I \sim p_{data}}\left[\text{BCE}(D(G(I)), 1)\right] + \lambda\, \mathbb{E}_{I \sim p_{data}}\left[\|G(I) - I\|_1\right] \tag{7}$$

where $D(G(I))$ is the discriminator prediction on the generated samples, $\text{BCE}(D(G(I)), 1)$ is the adversarial loss that encourages the generator to produce realistic images, $\|G(I) - I\|_1$ is the L1 reconstruction loss, and λ is a balancing term to balance adversarial and reconstruction objectives. The model optimisation is done using the Adam optimizer, as shown in equation 8.

$$\theta_G \leftarrow \theta_G - \eta \nabla_{\theta_G} L_G \tag{8}$$

where θ_G are the generator parameters, $\nabla_{\theta_G} L_G$ is the gradient of the generator loss with respect to its parameters, and η is the learning rate of the model

The algorithm 2 shows the step-by-step working of the process.

Algorithm 2 Neural ODE-Based Generator

Input: Thermal image I

Output: Generated image I_G

1: **Begin**

2: Extract the initial latent representation using the convolutional encoder.

$$h_0 = E(I, \theta_e)$$

3: Define the ODE transformation function.

$$\frac{dh}{dt} = f(h, t, \theta_{\text{ODE}})$$

4: Solve for the final latent state using numerical integration.

$$h_T = \text{ODEINT}(f, h_0, t, \text{method} ='' rk4'')$$

5: Reconstruct the image from the transformed latent representation.

$$I_G = f_{\text{dec}}(h_T, \theta_{\text{dec}})$$

6: Use the Tanh activation in the final layer, to ensure output values lie in the range $[-1, 1]$.

7: **End**

C. LIQUID NEURAL NETWORK-BASED DISCRIMINATOR

The LNN Discriminator presents a time-dependent method to differentiate generated images from the real images. The LNN-based model integrates continuous-time dynamics to improve feature representations over several time steps, unlike the traditional discriminators, which depend on static feature extraction. This model starts with an encoded representation derived from the convolutional layer extraction of spatial features from the input image, followed by a liquid state evolution under the control of an ODE. This representation is then iteratively generated via an ODE solver; the dynamics of the hidden state depend on both the input and a trainable decay parameter τ. After evolving through time, the final state is processed through a fully connected layer, and a sigmoid activation produces the probability score, determining whether the input is real or generated. This time-dependent feature refinement increases the discriminator's ability to capture complex representations, hence strengthening its resistance against adversarial generation. A detailed mathematical formulation of the LNN-based discriminator is given below:

1) IMAGE ENCODING AND FEATURE EXTRACTION

The input image, I which is either real or generated, is processed using a feature extractor:

$$x_{enc} = f_{enc}(I) \tag{9}$$

where $f_{enc}(.)$ is a Convolutional Neural Network (CNN) that extracts the spatial features. and x_{enc} is the encoded feature map, which serves as the initial state for the liquid neural network.

Algorithm 3 shows the step-by-step process of the discriminator.

Algorithm 3 Liquid Neural Network (LNN) Discriminator

Input: Thermal image I (Real or Generated)
Output: Discriminator score $D(I)$

1: **Begin**
2: Extract the spatial features from the input image using convolutional layers.

$$x_{\text{enc}} = f_{\text{enc}}(I)$$

3: Initialize the hidden state h_0 as a zero tensor.
4: **for** each time step t **do**
5: Compute the liquid state dynamics using an ODE solver.

$$\frac{dh}{dt} = \frac{1}{\tau}(f_{\text{liquid}}(h + x_{\text{enc}}) - h)$$

6: **end for**
7: Compute the final discriminator score using sigmoid activation.

$$D(I) = \sigma(f_{final}(h_T))$$

8: **End**

2) LIQUID NEURAL CORE

The Liquid Neural Core is an important part of the LNN-based discriminator that represents the evolution of the encoded features dynamically over a period of time using an ODE. Unlike the traditional discriminators that use a fixed sequence of layers, this approach dynamically modifies the feature representations as they get modified over time.

(a) **Why did we use the time-dependent feature evolution?** In a normal CNN-based discriminator, the feature representation remains static after the extraction of spatial features using convolutional layers. However, in real-world scenarios like heat diffusion in thermal images, the patterns evolve dynamically. By incorporating a time-dependent transformation, the discriminator continuously refines its feature representation, allowing it to more effectively capture the underlying structure of the input.

(b) **Mathematical Formulation of the Liquid Neural Core** The hidden representation evolves according to an ODE as shown in equation 10.

$$\frac{dh}{dt} = \frac{1}{\tau}(f_{\text{liquid}}(h + x_{\text{enc}}) - h) \tag{10}$$

where h is the hidden state that evolves over time, x_{enc} is the encoded feature map that is obtained from the convolutional layers, $f_{\text{liquid}}(h)$ a neural transformation function, τ a trainable decay parameter controlling how fast the hidden state updates, t is the continuous time variable over which the transformation occurs.

(c) **Why did we use an ODE for feature transformation?** In a standard neural network, transformations are applied in discrete steps layer by layer. However, for an ODE-based model, they continuously adjust the feature representations, giving a smoother feature evolution over time. The adaptive learning of the ODE model refines representations dynamically and provides better generalization by detecting patterns beyond fixed layers. In an LNN Discriminator, the liquid neural core replaces traditional recurrent structures like LSTMs or GRUs, making it more flexible and computationally efficient.

(d) **Using Runge-Kutta (RK4) for faster solutions** Most of the time, we cannot solve the ODE analytically, so we use a numerical solver to approximate the solution. The discriminator integrates the equation using the Runge-Kutta (RK4) method, a popular technique for solving ODEs with high accuracy. Equation 11 shows how to compute the hidden state after evolution.

$$h_T = ODEINT(f_{liquid}, h_0, t, method ='' rk4'') \tag{11}$$

where h_T is the final hidden state at time T, h_0 is the initial hidden state that is learnt dynamically, t is the time vector that defines discrete steps $t_0, t_1, \ldots t_T$, $ODEINT$ is an ODE solver from the torchdiffeq package, which uses the $RK4$ for stable integration. The RK4 method approximates the solution by evaluating the function at

four points and computing a weighted average as shown below.

$$h_{t+1} = h_t + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{12}$$

where k_1 is the initial slope at h_t, k_2&k_3 are intermediate slopes at half-steps, and k_4 is the final slope at $t+1$.

(e) **Classification** After the feature transformation is completed, the final evolved representation h_T is passed through a classification layer.

$$D(I) = \sigma\left(f_{\text{final}}(h_T)\right) \tag{13}$$

In equation 13, $D(I)$ is a probability score between 0 and 1, f_{final} is a fully connected layer, and $\sigma(.)$ the activation function.

Algorithm 4 Training Process of LiquidGAN (Image-to-Image Setup)

Input: Generator G, Discriminator D, Training dataset, Hyper-parameters (learning rate η, β_1, β_2, ϵ)
Output: Optimized Generator G and Discriminator D

1: **Begin**
2: Initialize G and D with random weights.
3: **for** each epoch **do**
4: **for** each batch of real images I **do**
5: Generate the fake images.

$$I_G = G(I)$$

6: Compute the discriminator loss.

$$L_D = \tfrac{1}{2}\Big(\text{BCE}(D(I), 1) + \text{BCE}(D(I_G), 0)\Big)$$

7: Compute the generator loss.

$$L_G = \text{BCE}(D(I_G), 1)$$

8: Clip the gradients of the discriminator D and generator G to prevent exploding gradients.

$$g \leftarrow \text{Clip}(g, -\gamma, \gamma)$$

where γ is the clipping threshold.
9: Update the parameters of the discriminator D and generator G using the Adam optimizer.
10: **end for**
11: Save the generated images and model checkpoints periodically.
12: **end for**
13: **End**

D. TRAINING AND OPTIMISATION

The LiquidGAN is trained using a two-step process, where the generator and discriminator are updated iteratively.

1) DISCRIMINATOR TRAINING

The discriminator is trained to distinguish the real images I from the generated images $G(I)$, where I is a real image. We used the Binary Cross-Entropy (BCE) loss.

$$\begin{aligned} L_D = {} & \tfrac{1}{2}\,\mathbb{E}_{I\sim p_{data}}[\text{BCE}(D(I), 1)] \\ & + \tfrac{1}{2}\,\mathbb{E}_{I\sim p_{data}}[\text{BCE}(D(G(I)), 0)] \end{aligned} \tag{14}$$

Here the first term maximizes the probability of correctly classifying real images, and the second term minimizes the probability of misclassifying generated images as real. The discriminator parameters are updated using the Adam optimizer.

2) GENERATOR TRAINING

The generator aims to fool the discriminator, i.e., generate images that will maximize the probability of being classified as real. The generator loss includes both adversarial loss (BCE loss) and the reconstruction loss (L1 loss).

(a) Adversarial loss (BCE loss)

$$L_G^{\text{adv}} = -\mathbb{E}_{I\sim p_{\text{data}}}\big[\log D(G(I))\big] \tag{15}$$

This encourages $G(I)$ producing images that are classified as real.

(b) Reconstruction Loss (L1 Loss)

$$L_{\text{recon}} = \mathbb{E}_{I\sim p_{\text{data}}}\big[\|G(I) - I\|_1\big] \tag{16}$$

This ensures the generated images retain structural similarity to real thermal images.

(c) Total Generator Loss

$$L_{\text{total}} = L_G^{\text{adv}} + \lambda L_{\text{recon}} \tag{17}$$

where λ is a weighting factor that balances adversarial learning and image fidelity. Adam Optimizer updates the parameters.

3) OPTIMISATION TECHNIQUE

Both the generator and the discriminator use the Adam optimizer.

$$\theta \leftarrow \theta - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{18}$$

where $\hat{m}_t$ and $\hat{v}_t$ are the first and second moment estimates, respectively. The gradient clipping is done to prevent exploding gradients. This method makes sure that LiquidGAN has stable convergence, good adversarial training, and the ability to keep the thermal image properties. Algorithm 4 shows the step-by-step training process of LiquidGAN.

V. EXPERIMENTAL SETUP

This section gives the hardware and software requirements, hyperparameter settings, and the model training information that were used while developing and evaluating our model LiquidGAN.

A. HARDWARE AND SOFTWARE REQUIREMENTS

All experiments including model training, ablation studies, and comparative assessments were conducted on Google Colab Pro+ cloud infrastructure. The software environment was set up with Python 3.10, PyTorch 2.1, CUDA 12.1, `torchdiffeq` library for Neural ODE integration.

In addition, we utilized Automatic Mixed Precision (AMP) and gradient scaling to accelerate training and reduce memory usage.

B. GENERATOR (NEURAL ODE) HYPERPARAMETERS

Refer to section II(A) to understand the structure of the generator, which integrates a Neural ODE module solved using a fixed-step fourth-order RK4 solver. RK4 was chosen over simpler options like Euler's method, because it provided a better compromise between numerical accuracy and computational cost.

The hyperparameters of the generator model were selected through a grid search process, using the validation Fréchet Inception Distance (FID) as the primary optimization criterion. The step size (Δt) for the ODE solver was set to 0.05, the latent dimension was fixed at 128, and the number of integration steps was chosen as 10. These settings were maintained consistently across all datasets to ensure training stability and comparability of results.

C. LNN BASED DISCRIMINATOR PARAMETER SETTINGS

The discriminator employs a LNN architecture, where neuron states evolve dynamically over time based on the differential equations.

The LNN discriminator comprises five convolutional layers for hierarchical feature extraction, followed by a Liquid Neural Core that manages the temporal state dynamics, and two fully connected layers for the decision-making. Hyperparameters for the LNN were tuned via grid search on validation FID score, with a decay parameter τ set to 0.6 (searched within $[0.1, 0.9]$) and a hidden state size of 128 dimensions. The incorporation of RK4 in the discriminator, significantly enhances the model's ability to learn temporally consistent and fine-grained thermal features. This improves the adversarial training stability and generation fidelity.

D. TRAINING CONFIGURATION

Both the generator and the discriminator were trained using the Adam optimizer with a learning rate of 0.0002, β_1 set to 0.5, and β_2 set to 0.999. The models were trained with a batch size of 32 for 100 epochs per dataset. The training time varied from 3.5 hrs to 4.5 hrs approximately for different datasets for 100 epochs. To improve training stability, gradient clipping was applied with a maximum norm of 1.0. Additionally, AMP was employed to accelerate convergence and optimize GPU memory usage.

E. DATASET PREPROCESSING

All datasets were normalized and resized to a consistent spatial resolution. PyTorch data loaders were used with `pin_memory=True` to enhance GPU data transfer speeds. Temperature normalization was applied to ensure thermal consistency across datasets.

VI. RESULTS AND ANALYSIS

The efficiency of the proposed methodology, integrating GANs, neural ODE, and LNNs, was evaluated using both qualitative and quantitative metrics and comparative studies with existing baseline GAN architectures like Standard Generative Adversarial Network, Deep Convolutional Generative Adversarial Network (DCGAN) [14], Wasserstein GAN-Gradient Penalty (WGAN-GP) [15].

A. DATASET DESCRIPTION

Table 1 provides a summary of the datasets used, including the number of images, resolution, and application domain.

TABLE 1. Summary of thermal imaging datasets with their attributes.

Dataset	# Images	Resolution	Domain
KAIST Multispectral [19]	9500	640×480	Surveillance
CBSR NIR Face Dataset [20]	3940	480x640	Human Identification
Neonatal Dataset [21]	11000	640x480	Healthcare
Sapota Dataset [22]	5030	640x480	Agriculture
Chilli Dataset [23]	3000	640x480	Agriculture

These datasets are from diverse applications, including surveillance, human identification, respiratory syndrome detection, Sapota bruise detection, and disease classification. In the section below, we calculate the quality of synthetic image generation, classifier performance, and computational efficiency.

B. QUALITATIVE ANALYSIS OF THE GENERATED IMAGES

We use a difference map to assess the qualitative performance of the generated images, emphasising the differences between the real and generated images. A completely black difference map says the generated image is identical to the real image pixelwise, but it does not necessarily indicate a perfect GAN. Figure 3 shows the real, generated and the difference map of 5 images taken from the chilli leaf dataset.

C. QUANTITATIVE ANALYSIS OF THE GENERATED IMAGES

The generator's ability to create high-quality synthetic thermal images was evaluated using the following metrics.

1) Frechet Inception Distance (FID)
2) Structural Similarity Index (SSIM)
3) Peak Signal-to-Noise Ratio (PSNR)

The Fréchet Inception Distance (FID) [9] measures the similarity between real and generated images in a feature space. Lower values say that the model has generated images that are closer to real samples in feature space. The FID score

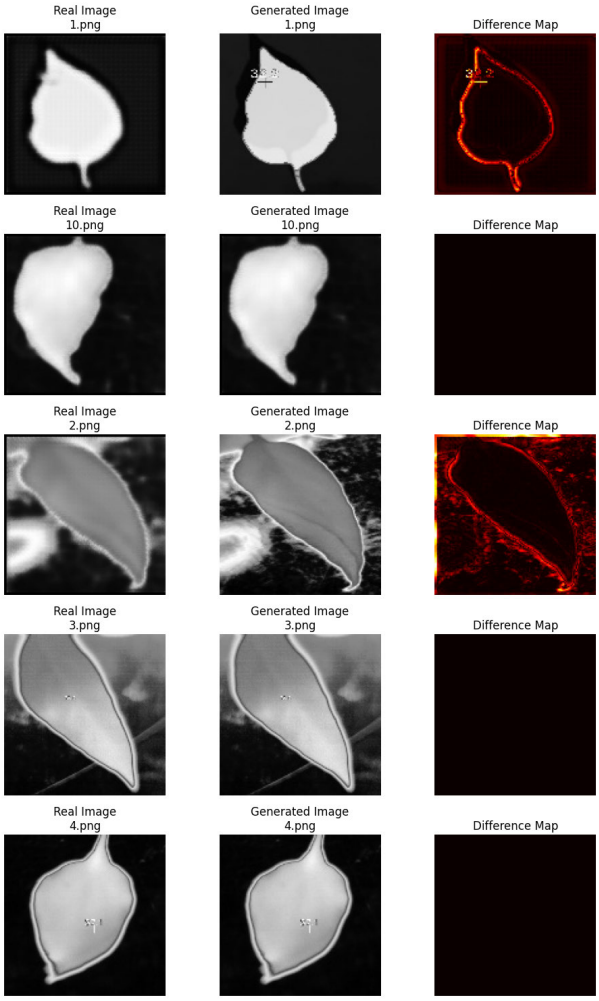

FIGURE 3. Comparison of real and generated images with their difference maps.

is calculated as given below.

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \text{Tr}\left(C_r + C_g - 2\left(C_r C_g\right)^{1/2}\right) \quad (19)$$

where μ_r and C_r represent the mean and covariance of the real images, and μ_g and C_g represent the mean and covariance of the generated images. $||.||_2^2$ is the squared euclidean distance between the mean and $Tr()$ represents the trace of the matrix.

Structural Similarity Index (SSIM) [11] is a metric that measures the similarity between two generated and real images. It compares the luminance, contrast, and structure of the images to provide a score between -1 and 1; the higher the SSIM, the superior the structural and perceptual quality in images. The SSIM score [11] is calculated as below.

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (20)$$

In this context, μ_x, μ_y denote the mean intensities of images x and y, respectively; σ_x^2, σ_y^2 signify the variances of x and y; σ_{xy} indicates the covariance between x and y; and C_1 and C_2 are minimal constants employed to stabilise the division.

By comparing the intensity differences between the real and generated images, the Peak Signal-to-Noise Ratio (PSNR) measures the image quality. The higher the PSNR value, the better the performance of the model. We calculate the PSNR as follows.

$$\text{PSNR} = 10 \cdot \log_{10}\left(\frac{MAX_I^2}{\text{MSE}}\right) \quad (21)$$

where MAX_I denotes the maximum pixel value of the image and MSE is the Mean Squared Error between the real and generated images.

Table 2 gives an overview of the FID, PSNR, and SSIM scores of the LiquidGAN model in different datasets used.

TABLE 2. FID, PSNR and SSIM Scores for different datasets.

Dataset	FID Score	PSNR(dB)	SSIM
KAIST Multispectral	20.34	31.22	0.88
CBSR NIR Face Dataset	27.81	27.40	0.82
Neonatal Dataset	23.90	30.20	0.84
Sapota Dataset	26.23	28.12	0.81
Chilli Dataset	20.70	30.89	0.87

D. COMPARATIVE ANALYSIS WITH BASELINE GAN MODELS

Table 3 gives a comparative evaluation of the proposed LiquidGAN against existing GAN architectures. We used DCGAN, StyleGAN, and WGAN-GP to compare our proposed model based on the key quantitative evaluation metrics we discussed so far. For all the datasets, the proposed method consistently shows the lowest FID values, indicating a better quality of the generated data. The highest PSNR values prove that the proposed model has a better reconstruction quality than other GANs, and the highest SSIM score shows the superior structural and perceptual quality of the generated images.

Conventional GAN models like DCGAN, WGAN-GP, and StyleGAN have certain limitations while generating thermal images. DCGAN produces blurry results and suffers from mode collapse because of its shallow architecture and fixed convolutional filters. WGAN-GP uses a different loss function to improve training stability but fails to model smooth temperature transitions because of its static feature extraction. StyleGAN, although more advanced, is mainly designed for RGB images but struggles to handle the continuous and subtle variations in thermal images. However, our model LiquidGAN uses an neural ODE generator that captures smooth, gradual changes in temperature and an LNN-based discriminator that adapts its internal state over time to better detect thermal patterns. As shown in Table 3,

TABLE 3. Comparison of FID, PSNR, and SSIM values across different GANs and the proposed methodology.

Dataset	DCGAN			StyleGAN			WGAN-GP			Proposed Model		
	FID	PSNR(dB)	SSIM	FID	PSNR(dB)	SSIM	FID	PSNR(dB)	SSIM	FID	PSNR(dB)	SSIM
KAIST Multispectral	32.67	24.12	0.76	27.56	26.23	0.81	23.45	27.89	0.85	20.34	31.22	0.88
CBSR NIR Face Dataset	40.23	22.34	0.72	35.12	23.45	0.79	30.78	26.34	0.78	27.81	27.40	0.82
Neonatal Dataset	35.12	23.45	0.78	28.34	25.67	0.82	24.78	27.45	0.86	23.90	30.20	0.84
Sapota Dataset	38.45	23.12	0.74	33.67	24.89	0.78	28.45	26.45	0.84	26.23	28.12	0.81
Chilli Dataset	36.12	22.89	0.73	32.45	24.34	0.76	28.12	25.98	0.80	20.70	30.89	0.87

LiquidGAN consistently outperforms other models. For instance, on the Chilli dataset, it achieves the lowest FID of 20.70 and the highest PSNR and SSIM values of 30.89 dB and 0.87, respectively. This shows LiquidGAN ability to generate more realistic and thermally accurate images.

E. ABLATION STUDY

To see how different parts of the proposed model LiquidGAN affect each other, we did an ablation study where we took out important parts and watched how the model's performance went down. Table 4 shows that the ODE solver contributes more towards the feature smoothness in the generator; the ODE module is removed and replaced with a simplified DCGAN-style generator composed of transposed convolutional layers. This structure lacks the continuous-time dynamics modeled by ODEs and serves as a shallow, non-dynamic baseline for comparison.

The LNN-based discriminator is generally used to enhance the model's ability to distinguish the fine details in thermal images. Since it has time-dependent feature transformations, it makes it harder for the generator to fool the discriminator. Without it, the discriminator relies solely on static spatial features, leading to a less robust and adaptive learning process. Removing it also eliminates the dynamic adaptability of the liquid neural network components, which were crucial in detecting subtle inconsistencies in the generated images.

Removing both components leads to a sharp increase in FID and degradation in perceptual quality. This is because the neural ODE generator enables smooth and continuous transformations in the latent space. Without it, the generator will go back to a fixed-layer CNN approach, which is less expressive in capturing complex variations. This makes the generated images less diverse and less realistic. Furthermore, if you take out the LNN-based discriminator, you can't use differential equations to improve feature representations in real time. Instead, you have to use a static CNN, which doesn't have time-dependent learning. As a result, the discriminator becomes easier to fool over time. We also tested a version where the ODE solver was replaced with a regular feedforward neural network (FFN). While this made training simpler, it led to lower performance across all metrics, confirming the advantage of using continuous-time transformations.

TABLE 4. Ablation study: impact of different components in LiquidGAN.

Model Variant	FID	SSIM	PSNR
Full LiquidGAN	**20.34**	**0.88**	**31.22**
Without ODE Solver	29.32	0.83	21.56
Without LNN Discriminator	31.12	0.79	19.87
With FFN Instead of ODE	27.89	0.81	22.75
Without Both	37.45	0.74	17.43

F. DISCUSSION

LiquidGAN has a strong potential in various real-world thermal imaging applications where data is often limited or expensive to collect. In healthcare, especially neonatal monitoring, synthetic thermal images can be used to augment rare pathological cases or respiratory syndromes, improving early diagnosis models. In agriculture, LiquidGAN-generated data can help train models to detect subtle signs of crop stress or plant diseases using thermal imagery. For security and surveillance systems operating in low-light conditions, LiquidGAN can improve training datasets for person detection or anomaly recognition. Moreover, the architecture can be adapted for specific domains by fine-tuning the model on small, domain-specific datasets or by integrating multi-modal data such as RGB and infrared to enhance contextual learning. Future extensions may also include optimizing the generator and discriminator for edge-device deployment by reducing computational overhead, or dynamically adjusting ODE solver parameters based on image resolution and complexity. These adaptations will enable LiquidGAN to serve as a versatile generative framework for diverse thermal imaging applications.

VII. CONCLUSION

Our paper proposed LiquidGAN, a novel generative adversarial architecture that integrates neural ODEs into the generator and LNNs into the discriminator to address the challenges of thermal image synthesis under data scarcity. By adding continuous-time transformations through the ODEs and adaptive feature extraction using the LNN-based discriminator, our model effectively captures complex image distributions while maintaining model stability in adversarial training.

Unlike the traditional GANs that applied fixed transformations, our model with the neural ODE generator learned continuous latent space dynamics, improving the smoothness and realism of the generated thermal images. This procedure allowed better data diversity, addressing the limitations of conventional augmentation techniques. Also, the LNN-based discriminator modelled feature evolution over time, making it more robust to mode collapse and improving generalisation when distinguishing real and generated images.

Quantitative evaluations demonstrated superior performance over baseline GANs in terms of FID, PSNR, and SSIM metrics on various thermal datasets. As a future work, the LiquidGAN can be extended for visible-to-infrared image synthesis and multi-spectral image fusion in the context of cross-modality adaptation. Adding continuous-time neural representations, adaptive learning, and ODE-driven generative processes to LiquidGAN makes it a better and more useful way to improve thermal images.

REFERENCES

[1] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," 2014, *arXiv:1406.2661*.

[2] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein GAN," 2017, *arXiv:1701.07875*.

[3] T. Karras, S. Laine, and T. Aila, "A style-based generator architecture for generative adversarial networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 4396–4405, doi: 10.1109/CVPR.2019.00453.

[4] V. A. Mizginov, V. V. Kniaz, and N. A. Fomin, "A method for synthesizing thermal images using GAN multi-layered approach," *Int. Arch. Photogramm., Remote Sens. Spatial Inf. Sci.*, vol. 44, pp. 155–162, Apr. 2021.

[5] S. Sarath and J. J. Nair, "Detection and classification of respiratory syndromes in original and modified DCGAN augmented neonatal infrared datasets," *Proc. Comput. Sci.*, vol. 233, pp. 422–431, Jan. 2024.

[6] T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, "Neural ordinary differential equations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 6571–6583.

[7] W. Grathwohl, R. T. Q. Chen, J. Bettencourt, I. Sutskever, and D. Duvenaud, "FFJORD: Free-form continuous dynamics for scalable reversible generative models," 2018, *arXiv:1810.01367*.

[8] R. Hasani, M. Lechner, A. Amini, D. Rus, and R. Grosu, "Liquid time-constant networks," 2020, *arXiv:2006.04439*.

[9] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," 2017, *arXiv:1706.08500*.

[10] S. Navaneeth, S. Sarath, B. Amba Nair, K. Harikrishnan, and P. Prajal, "A deep-learning approach to find respiratory syndromes in infants using thermal imaging," in *Proc. Int. Conf. Commun. Signal Process. (ICCSP)*, Chennai, India, Jul. 2020, pp. 0498–0501, doi: 10.1109/ICCSP48568.2020.9182231.

[11] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, Apr. 2004, doi: 10.1109/TIP.2003.819861.

[12] N. Aloysius and M. Geetha, "A review on deep convolutional neural networks," in *Proc. Int. Conf. Commun. Signal Process. (ICCSP)*, Apr. 2017, pp. 0588–0592.

[13] V. Mohan, M. P. Hariprasad, and V. Menon, "FringeNet: A cyclic U-Net model with continuity imposed hybrid cyclic loss for demodulation of isochromatics in digital photoelasticity," *Opt. Lasers Eng.*, vol. 181, Oct. 2024, Art. no. 108428.

[14] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," 2015, *arXiv:1511.06434*.

[15] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville, "Improved training of Wasserstein GANs," vol. 30, 2017, pp. 1–11.

[16] N. Aloysius and M. Geetha, "A scale space model of weighted average CNN ensemble for ASL fingerspelling recognition," *Int. J. Comput. Sci. Eng.*, vol. 22, no. 1, pp. 154–161, 2020.

[17] S. Remya, T. Anjali, S. Abhishek, S. Ramasubbareddy, and Y. Cho, "The power of vision transformers and acoustic sensors for cotton pest detection," *IEEE Open J. Comput. Soc.*, vol. 5, pp. 356–367, 2024.

[18] X. Sun, C. Chen, X. Wang, J. Dong, H. Zhou, and S. Chen, "Gaussian dynamic convolution for efficient single-image segmentation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 5, pp. 2937–2948, May 2022.

[19] S. Hwang, J. Park, N. Kim, Y. Choi, and I. S. Kweon, "Multispectral pedestrian detection: Benchmark dataset and baseline," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1037–1045.

[20] S. Z. Li, R. Chu, S. Liao, and L. Zhang, "Illumination invariant face recognition using near-infrared images," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 29, no. 4, pp. 627–639, Apr. 2007.

[21] C. B. Pereira, X. Yu, T. Goos, I. Reiss, T. Orlikowsky, K. Heimann, B. Venema, V. Blazek, S. Leonhardt, and D. Teichmann, "Noncontact monitoring of respiratory rate in newborn infants using thermal imaging," *IEEE Trans. Biomed. Eng.*, vol. 66, no. 4, pp. 1105–1114, Apr. 2019.

[22] M. Raviteja, V. V. V. K. Reddy, S. Aqibuddin, M. T. Soppa, and S. Sarath, "Sapota quality: Bruise detection and shelf life prediction," in *Proc. 5th Int. Conf. Electron. Sustain. Commun. Syst. (ICESC)*, Coimbatore, India, Aug. 2024, pp. 1469–1474.

[23] A. T. George, D. M. Deepak, G. P. Kunnath, G. P. Devi, and S. Sarath, "Deep learning-driven low-resolution thermal image reconstruction for enhanced CWSI analysis," in *Proc. 5th Int. Conf. Electron. Sustain. Commun. Syst. (ICESC)*, Coimbatore, India, Aug. 2024, pp. 1571–1576.

S. SARATH received the M.Tech. degree in computer science and engineering from M. S. University, India, in 2010. He is currently a Senior Assistant Professor with the School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri. He has written research papers for a number of reputable conferences and journals, such as IEEE, ACM, and Springer. His recent research centres on deep learning-based precision agriculture, non-invasive health monitoring, infant thermal image enhancement, autonomous systems, and deception detection. He has been actively involved in the development of novel methods for thermal image-based recognition, real-time surveillance, and environmental monitoring using state-of-the-art machine learning and deep learning techniques. His research interests include thermal image processing, computer vision, deep learning, medical image analysis, precision agriculture, and UAV detection.

JYOTHISHA J. NAIR (Senior Member, IEEE) received the M.Tech. degree in computer science and engineering, specializing in image processing, and the Ph.D. degree from the National Institute of Technology (NIT) Calicut, India. She has authored numerous research papers in various fields, attracting numerous citations for her work. She has also contributed to the development of interactive learning systems for the hearing impaired and vocally challenged and has researched federated learning frameworks for privacy-preserving diagnostics. Her research interests include deep learning, complex network analysis, computer vision, and medical image analysis.